CSCI 271: Assignment 4 — Iteration & Loop Patterns

Part 1: Multiple Choice (8 Questions · 8 points)

1. In a while loop, when is the condition checked?

☐ A) After the loop body runs, but not before ☐ B) Before every single pass, including the first

☐ C) Only once, before the loop starts ☐ D) Only after the loop has already ended

2. Which loop pattern do you use when you don't know in advance how many values the user will enter?

☐ A) Counter-controlled ☐ B) Sentinel-controlled

☐ C) do...while only ☐ D) None of these

3. In a sentinel loop, why do we check if (count != 0) before dividing to get an average?

☐ A) To silence a compiler warning

☐ B) To avoid dividing by zero when no values were entered

☐ C) To make the loop run faster

☐ D) It's required syntax for every while loop

4. Which for loop header correctly prints the numbers 1 through 5?

☐ A) for (int i = 0; i < 5; i++) ☐ B) for (int i = 1; i <= 5; i++)

☐ C) for (int i = 1; i < 5; i++) ☐ D) for (int i = 5; i <= 1; i++)

5. What is different about a do...while loop compared to a while loop?

☐ A) do...while never uses a condition

☐ B) do...while's body always runs at least once

☐ C) do...while can only count downward

☐ D) do...while doesn't require a semicolon anywhere

6. What does continue; do inside a loop?

☐ A) Immediately exits the loop entirely

☐ B) Skips the rest of the current pass and moves to the next condition check

☐ C) Restarts the whole program

☐ D) Does the exact same thing as break;

7. Which mistake causes an infinite loop?

☐ A) Using <= instead of < in the condition

☐ B) Forgetting to update the loop's counter or condition variable

☐ C) Declaring the counter as an int

☐ D) Using a for loop instead of a while loop

8. Given: int count = 0; for (int i = 1; i <= 10; i += 2) { count++; } What is count after the loop? [DAY 2] *(5 pt)*

☐ A) 10 ☐ B) 5 ☐ C) 9 ☐ D) 2

Part 2: Coding Problems (22 points)

Problem 1: Multiplication Table (10 pts)

Write a C++ program called mult_table.cpp that prints a multiplication table for a number the user enters.

☐ Ask the user to enter a whole number using cin.

☐ Use a for loop to multiply that number by 1 through 10.

- ☐ Print one line per multiplication, e.g. “7 x 1 = 7”, all the way through “7 x 10 = 70”.

```cpp
```

```
#include <iostream>
using namespace std;
int main() {
    int number;
    cout << "Enter a number: ";
    cin >> number;
    for (int i = 1; i <= 10; i++) {
        cout << number << " x " << i << " = " << number * i << endl;
    }
    return 0;
}
```

Example interaction:

```
Enter a number: 7
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
...
7 x 10 = 70
```

Problem 2: Menu Choice Validation (12 pts)

Write a C++ program called menu_validate.cpp that asks the user to enter a menu choice and won't move on until they enter a valid one.

- ☐ Use a do...while loop, since the prompt must show at least once no matter what.
- ☐ Ask the user to enter a choice: 1, 2, or 3.
- ☐ If the number entered is NOT 1, 2, or 3, print “Invalid choice, try again.” and re-prompt.
- ☐ Once a valid choice is entered, print “You selected option X.” with the number they chose.

```cpp
#include <iostream>
using namespace std;
int main() {
    int choice;
    do {
        cout << "Enter a menu choice (1, 2, or 3): ";
        cin >> choice;
        if (choice < 1 || choice > 3) {
            cout << "Invalid choice, try again." << endl;
        }
    } while (choice < 1 || choice > 3);
    cout << "You selected option " << choice << "." << endl;
    return 0;
}
```

```

Example interaction:

```
Enter a menu choice (1, 2, or 3): 5
Invalid choice, try again.
Enter a menu choice (1, 2, or 3): 0
Invalid choice, try again.
Enter a menu choice (1, 2, or 3): 2
You selected option 2.
```

Explaining code line by line

1. `#include <iostream>`

This gives your program the tools to use `cout` and `cin`.

- `cout` = output → puts something on the screen
- `cin` = input → gets something from the user

`using namespace std;`

This lets us write `cout` and `cin` instead of having to write `std::cout` and `std::cin`

`int main() {`

This is where the program starts running.

`int number;`

We create a variable called `number`.

Because we use `int`, `number` will hold a whole number, such as:

7
10
25
100

`cout << "Enter a number: ";`

This displays:

Enter a number:

It's telling the user what to do.

`cin >> number;`
```

This waits for the user to type a number.

If the user types:

7

then the number becomes 7.

Remember:

> `cin` = get information from the user

The loop

for (int i = 1; i <= 10; i++) {

This is the most important part.

It means:

> Start `i` at 1, keep going while `i` is 10 or less, and increase `i` by 1 each time.

So `it becomes`:

1
2
3
4
5
6
7
8
9
10

You can think of it as:

Start → Check → Run → Increase → Check again

```
cout << number << " x " << i << " = " << number * i << endl;
```

This prints the multiplication.

If `number` is 7 and `i` is 1:

7 x 1 = 7

Then `i` becomes 2:

7 x 2 = 14

Then 3:

7 x 3 = 21

And so on until 10.

The important part is:

number * i

That's actually doing the multiplication.

`}`

This ends the `for` loop.

`return 0;`

This tells the computer:

The program finished

`}`

This ends the `main` function and therefore ends the program.

2. The `menu_validate.cpp` program asks the user to enter a menu choice from 1 to 3. It uses a `do while` loop because the program needs to ask for a choice at least once. The `cin` statement stores the user's choice in the `choice` variable. The `if` statement checks if the choice is less than 1 or greater than 3. If it is invalid, the program prints “Invalid choice, try again.” and asks the user again. The loop continues until the user enters 1, 2, or 3. Once the user enters a valid choice, the program prints which option they selected.